Linked List - Step by Step Working

Sneha ke handwritten notes ke hisaab se: 28 codes, 5 sections. Har code mein dekho kaunsa pointer (head, tail, temp) kab update hota hai aur delete kab hota hai.

Pehchano kaunsa pattern: (1) ek hi pass mein kuch dhundna = `slow/fast`, (2) links ulte karne = `prev/curr/next`, (3) node jodna/hatana = `temp/head/tail`. Baaki sab inhi ka mix hai.

Section 1: Singly LL (7 codes)

1. Push front (O(1))

1. `newNode = new Node(val)` banao
2. Agar `head == NULL` (list empty): `head = tail = newNode`, phir `return`
3. Warna: `newNode->next = head` (naya node purane head ko point kare)
4. `head = newNode` (head ab naya node)
5. `tail` ko touch nahi karna

2. Push back (O(1))

1. `newNode = new Node(val)` banao
2. Agar `head == NULL`: `head = tail = newNode` (return nahi, aage else hai)
3. Warna: `tail->next = newNode` (purana last naye ko point kare)
4. `tail = newNode` (tail ab naya node)
5. `head` ko touch nahi karna

3. Print / traverse (O(n))

1. `temp = head`
2. Jab tak `temp != NULL`:
   - `cout << temp->data`
   - `temp = temp->next`
3. Loop ke baad `cout << NULL`

4. Pop front (updated) (O(1))

1. Agar `head == NULL`: "LL empty" print karo, `return`
2. `temp = head` (pehla node save)
3. `head = head->next` (head aage khisak gaya)
4. `temp->next = NULL` (purana node alag)
5. `delete temp`
6. **Naya:** agar `head == NULL` (sirf 1 node thi) to `tail = NULL`

Edge case: bina step 6 ke tail deleted node ko point karta rahega (dangling pointer).

5. Pop back (updated) (O(n))

1. Agar `head == NULL`: "LL empty" print karo, `return`
2. **Naya:** agar `head == tail` (sirf 1 node): `delete head`, `head = tail = NULL`, `return`
3. `temp = head`
4. Jab tak `temp->next != tail`: `temp = temp->next` (temp second-last par aayega)
5. `delete tail`
6. `tail = temp` (tail ab second-last)
7. `tail->next = NULL`

Edge case: single node mein bina step 2 ke loop temp ko NULL tak le jata aur crash hota.

6. Insert in middle at pos (O(n))

1. Agar `pos < 0 || head == NULL`: "Invalid pos", `return`
2. Agar `pos == 0`: `push_front(val)`, `return`
3. `temp = head`
4. `for (i = 0; i < pos-1; i++)`: `temp = temp->next` (temp pos se ek pehle wale node par)
5. Agar `temp == NULL || temp->next == NULL`: "Invalid", `return`
6. `newN = new Node(val)`
7. `newN->next = temp->next` (**pehle** naye ko aage wale se jodo)
8. `temp->next = newN` (**phir** pichle ko naye se jodo)

Step 7 aur 8 ka order ulta kiya to aage wali list kho jayegi.

7. Delete at index (O(n))

1. `temp` ko `head` se `pos-1` tak le jao
2. Agar `temp == NULL || temp->next == NULL`: "Invalid", `return`
3. `delNode = temp->next` (jo delete karna hai wo save)
4. `temp->next = delNode->next` (bypass)
5. `delete delNode`

Notes ka example: `ll.delete(1)` index 1 wala node delete karta hai.

Section 2: Pointer technique wale (8 codes)

8. Reverse LL (O(n))

1. `prev = NULL`, `curr = head`, `next = NULL`
2. Jab tak `curr != NULL`:
   - `next = curr->next` (aage wala pehle save)
   - `curr->next = prev` (link ulta)
   - `prev = curr`
   - `curr = next`
3. `return prev` (prev naya head hai)

9. Find middle node (O(n))

1. `slow = head`, `fast = head`
2. Jab tak `fast != NULL && fast->next != NULL`:
   - `slow = slow->next` (1 step)
   - `fast = fast->next->next` (2 step)
3. `return slow`

10. Detect cycle (O(n))

1. `slow = head`, `fast = head`
2. Jab tak `fast != NULL && fast->next != NULL`:
   - `slow = slow->next`, `fast = fast->next->next`
   - Agar `slow == fast`: `return true`
3. Loop ke baad `return false`

11. Cycle start dhundna (O(n))

1. Detect cycle wala code chalao. `slow == fast` mile to `isCycle = true`, `break`
2. Agar `isCycle == false`: `return NULL`
3. `slow = head` (fast meeting point par hi rehta hai)
4. Jab tak `slow != fast`: `slow = slow->next`, `fast = fast->next` (dono 1-1 step)
5. `return slow` (jahan milen wahi cycle ka start)

12. Remove cycle (O(n))

1. Pehle cycle detect karo. `isCycle == false` to `return`
2. `slow = head`, `prev = NULL`
3. Jab tak `slow != fast`:
   - `slow = slow->next`
   - `prev = fast` (fast badhane se **pehle** save)
   - `fast = fast->next`
4. `prev->next = NULL` (cycle ka last link tod diya)

Edge case: cycle head se hi shuru ho to loop chalta nahi, prev NULL rehta hai. Tab `while (fast->next != slow) fast = fast->next;` phir `fast->next = NULL`.

13. Merge two sorted LL (O(n+m))

Recursive:

1. Agar `l1 == NULL`: `return l2`. Agar `l2 == NULL`: `return l1`
2. Agar `l1->val <= l2->val`:
   - `l1->next = merge(l1->next, l2)`, `return l1`
3. Warna:
   - `l2->next = merge(l1, l2->next)`, `return l2`

Dummy + tail wala:

4. `dummy(0)`, `tail = &dummy`
5. Jab tak `l1 && l2`:
   - Agar `l1->val <= l2->val`: `tail->next = l1`, `l1 = l1->next`
   - Warna: `tail->next = l2`, `l2 = l2->next`
   - `tail = tail->next`
6. `tail->next = l1 ? l1 : l2` (bacha hissa jodo)
7. `return dummy.next`

14. Sort LL (merge sort) (O(n log n))

1. Agar `head == NULL || head->next == NULL`: `return head`
2. `slow = head`, `fast = head->next`
3. Jab tak `fast && fast->next`: `slow = slow->next`, `fast = fast->next->next`
4. `right = slow->next` (doosra half)
5. `slow->next = NULL` (list do hisson mein tooti)
6. `return merge(sort(head), sort(right))`

15. Copy list with random pointer (O(n), map)

1. Agar `head == NULL`: `return NULL`
2. `map<Node*, Node*> m` banao
3. `newHead = new Node(head->val)`, `oldTemp = head`, `newTemp = newHead`, `m[head] = newHead`
4. **Pehla pass:** jab tak `oldTemp->next != NULL`:
   - `copyN = new Node(oldTemp->next->val)`
   - `m[oldTemp->next] = copyN`
   - `newTemp->next = copyN`
   - `oldTemp = oldTemp->next`, `newTemp = newTemp->next`
5. `oldTemp = head`, `newTemp = newHead` (wapas start)
6. **Doosra pass:** jab tak `oldTemp != NULL`:
   - `newTemp->random = m[oldTemp->random]`
   - `oldTemp = oldTemp->next`, `newTemp = newTemp->next`
7. `return newHead`

#10, #11, #12 mein detect cycle ka hi code hai. #14 mein #9 (middle) aur #13 (merge) use hote hain.

Section 3: Doubly LL (5 codes)

Har node mein `data`, `next`, `prev`. Class mein `head`, `tail`. Har link **do taraf** jodna hai.

16. Print (O(n))

1. `temp = head`
2. Jab tak `temp != NULL`:
   - `cout << temp->data << " <=> "`
   - `temp = temp->next`
3. Loop ke baad `cout << NULL`

17. Push front (O(1))

1. `newN = new Node(val)` banao
2. Agar `head == NULL`: `head = tail = newN`
3. Warna:
   - `newN->next = head`
   - `head->prev = newN`
   - `head = newN`
4. `tail` ko touch nahi karna

18. Push back (O(1))

1. `newN = new Node(val)` banao
2. Agar `head == NULL`: `head = tail = newN`
3. Warna:
   - `newN->prev = tail`
   - `tail->next = newN`
   - `tail = newN`
4. `head` ko touch nahi karna

19. Pop front (O(1))

1. Agar `head == NULL`: "LL empty", `return`
2. `temp = head`
3. `head = head->next`
4. Agar `head != NULL`: `head->prev = NULL`
5. `temp->next = NULL`
6. `delete temp`

Edge case: sirf 1 node thi to `if (head == NULL) tail = NULL;` zaroor lagao.

20. Pop back (O(1))

1. Agar `head == NULL`: "LL empty", `return`
2. `temp = tail`
3. `tail = tail->prev`
4. Agar `tail != NULL`: `tail->next = NULL`
5. `temp->prev = NULL`
6. `delete temp`

Edge case: sirf 1 node thi to `if (tail == NULL) head = NULL;`. Singly mein pop back O(n) tha, doubly mein `tail->prev` se seedha O(1).

Section 4: Circular LL (5 codes)

Hamesha `tail->next = head` maintain karna hai. Notes mein `h` = head, `t` = tail. Print mein `temp != NULL` nahi chalega, `temp != h` lagta hai.

21. Insert at head (O(1))

1. `newN = new Node(val)` banao
2. **Base case:** agar `h == NULL`: `head = tail = newN`, `tail->next = h`, `return`
3. Warna (approach 1):
   - `newN->next = head`
   - `h = newN`
   - `tail->next = h`

Approach 2 (notes): `newN->next = t->next`, `t->next = newN`, `h = newN`. Result same.

22. Insert at tail (O(1))

1. `newN = new Node(val)` banao
2. **Base case:** agar `h == NULL`: `head = tail = newN`, `tail->next = h`, `return`
3. Warna:
   - `newN->next = h`
   - `t->next = newN`
   - `t = newN`
4. `head` ko touch nahi karna

23. Print (O(n))

1. Agar `h == NULL`: `return`
2. `cout << h->data << "->"` (head pehle alag)
3. `temp = h->next`
4. Jab tak `temp != h`:
   - `cout << temp->data << "->"`
   - `temp = temp->next`
5. Loop ke baad `cout << temp->data` (head dobara dikhta hai, list gol hai)

24. Delete at head (O(1))

1. **Case 1, empty:** agar `h == NULL`: `return`
2. **Case 2, single node:** agar `h == tail`: `delete h`, `h = t = NULL`
3. **Case 3, more nodes:**
   - `temp = h`
   - `h = h->next`
   - `t->next = h` (ye step mat bhoolna)
   - `temp->next = NULL`
   - `delete temp`

25. Delete at tail (O(n))

1. **Case 1, empty:** agar `h == NULL`: `return`
2. **Case 2, single node:** agar `h == tail`: `delete h`, `h = t = NULL`, `return`
3. **Case 3, more nodes:**
   - `temp = tail`
   - `prev = h`
   - Jab tak `prev->next != t`: `prev = prev->next` (prev second-last par)
   - `tail = prev`
   - `tail->next = h`
   - `temp->next = NULL`
   - `delete temp`

Har insert/delete ke baad poochho: `tail->next == head` abhi bhi hai?

Section 5: 3 Questions (3 codes)

26. Flatten multilevel DLL (O(n))

1. Agar `h == NULL`: `return h`
2. `curr = h`
3. Jab tak `curr != NULL`:
   - Agar `curr->child != NULL`:
   - `next = curr->next` (aage wala pehle save)
   - `curr->next = flatten(curr->child)`
   - `curr->next->prev = curr`
   - `curr->child = NULL`
   - **Tail dhundo:** jab tak `curr->next != NULL`: `curr = curr->next`
   - **Attach:** agar `next != NULL`: `curr->next = next`, `next->prev = curr`
   - `curr = curr->next`
4. `return h`

Idea: child list ko `curr` aur `curr->next` ke beech daalo.

27. Reverse K nodes (O(n))

1. **Check:** `temp = h`, `count = 0`. Jab tak `count < k`: agar `temp == NULL` to `return h`, warna `temp = temp->next`, `count++`
2. `prevN = reverseK(temp, k)` (baaki list recursion se reverse)
3. `temp = h`, `count = 0`
4. Jab tak `count < k`:
   - `next = temp->next`
   - `temp->next = prevN`
   - `prevN = temp`
   - `temp = next`, `count++`
5. `return prevN` (is group ka naya head)

Ye normal reverse (#8) hi hai, bas prev NULL ki jagah reversed baaki list se shuru hota hai.

28. Swap nodes in pairs (O(n))

1. Agar `head == NULL || head->next == NULL`: `return head`
2. `first = head`, `sec = head->next`, `prev = NULL`
3. Jab tak `first != NULL && sec != NULL`:
   - `third = sec->next` (agla pair save)
   - `sec->next = first`
   - `first->next = third`
   - Agar `prev != NULL`: `prev->next = sec`, warna `head = sec`
   - `prev = first`
   - `first = third`
   - Agar `third != NULL`: `sec = third->next`, warna `sec = NULL`
4. `return head`

`prev->next = sec` bhoolo mat, warna swapped pairs aapas mein jud nahi paayenge.